

Report Gestore dei Processi nei Sistemi Operativi

Consegna S3L1

1. Obiettivo

L'esercizio verte sui meccanismi di pianificazione dell'utilizzo della CPU. Dati 4 processi (P1, P2, P3, P4), individueremo il modo più efficace per la gestione e l'esecuzione dei processi. Abbozzeremo un diagramma che abbia sulle ascisse il tempo passato da un istante «0» e sulle ordinate il nome del Processo (P1, P2, P3, P4).

2. Dati dei Processi

I 4 processi arrivano alla CPU nell'ordine P1, P2, P3, P4. La tabella seguente riassume i tempi di esecuzione e di attesa I/O per ciascun processo.

Processo	Tempo di esecuzione	Tempo di attesa I/O	Esecuzione dopo attesa
P1	3 secondi	2 secondi	1 secondo
P2	2 secondi	1 secondo	—
P3	1 secondo	—	—
P4	4 secondi	1 secondo	—

3. Metodi di Scheduling Analizzati

Nella lezione teorica abbiamo parlato della gestione dei processi. Di seguito abbiamo una tabella che stabilisce la distinzione, tra le varie classificazioni dei OS, in base alla gestione delle operazioni e delle risorse

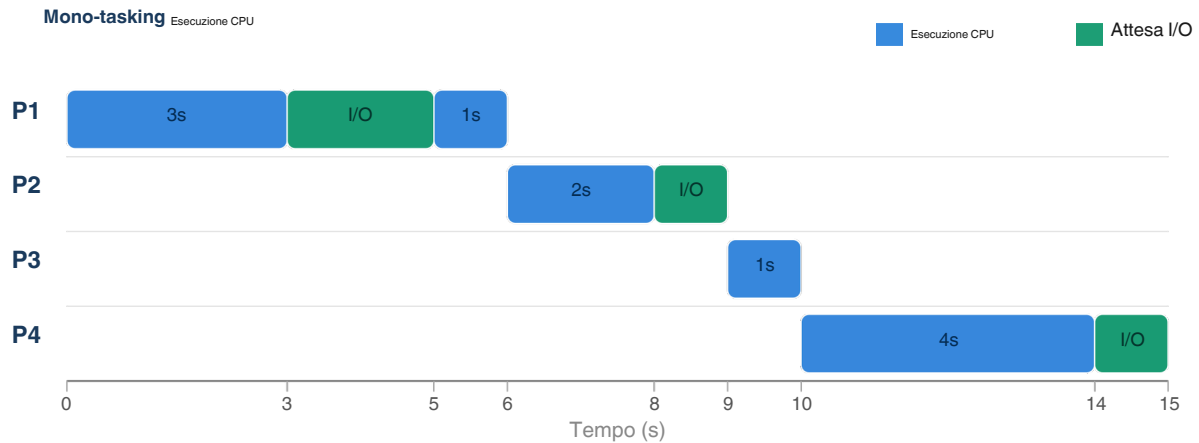
Metodo	Descrizione
Mono-tasking	Un solo processo alla volta occupa la CPU. Durante le attese I/O la CPU rimane completamente idle, causando spreco di risorse.
Multi-tasking	Quando un processo entra in attesa I/O, la CPU viene immediatamente assegnata al processo successivo. La CPU non è mai idle: i tempi di attesa di un processo vengono "riempiti" dall'esecuzione degli altri.
Time-sharing	Ogni processo riceve un quanto di tempo fisso (es. 1 secondo) in modo ciclico. La CPU alterna tra tutti i processi indipendentemente dallo stato I/O, dando l'impressione di esecuzione parallela.

4. Diagrammi di Gantt

I seguenti diagrammi mostrano l'esecuzione dei 4 processi secondo ciascun metodo. In blu i blocchi di esecuzione CPU, in verde i periodi di attesa I/O.

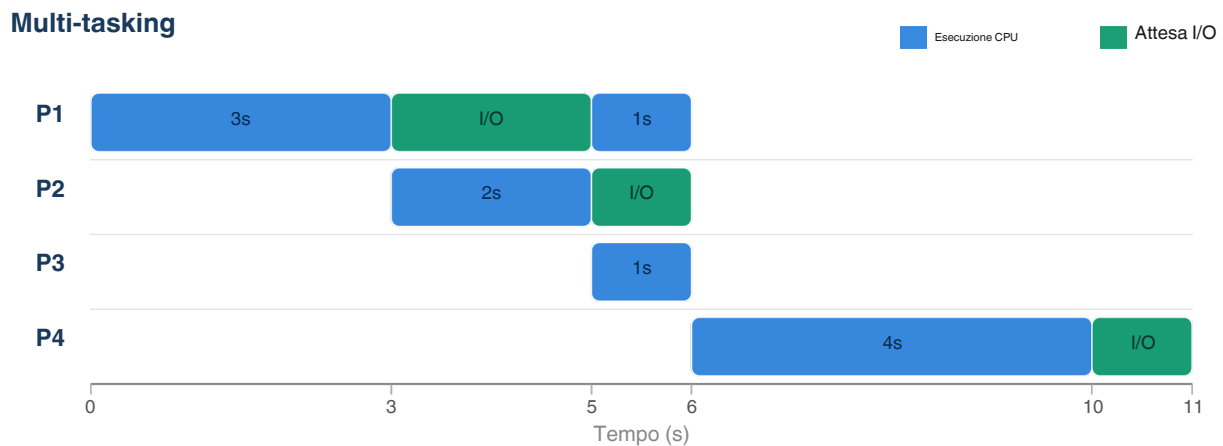
4.1 — Mono-tasking (totale: 15 secondi)

I processi vengono eseguiti uno alla volta in sequenza. La CPU resta in *IDLE* durante ogni attesa I/O.



4.2 — Multi-tasking (totale: 11 secondi)

Quando P1 va in *IDLE* I/O al secondo 3, la CPU passa subito a P2. Quando P2 va in *IDLE* I/O al secondo 5, la CPU esegue P3. Al termine di P3, torna P1 dal suo I/O e completa. Poi P4 inizia e viene interrotto dalla sua attesa I/O.

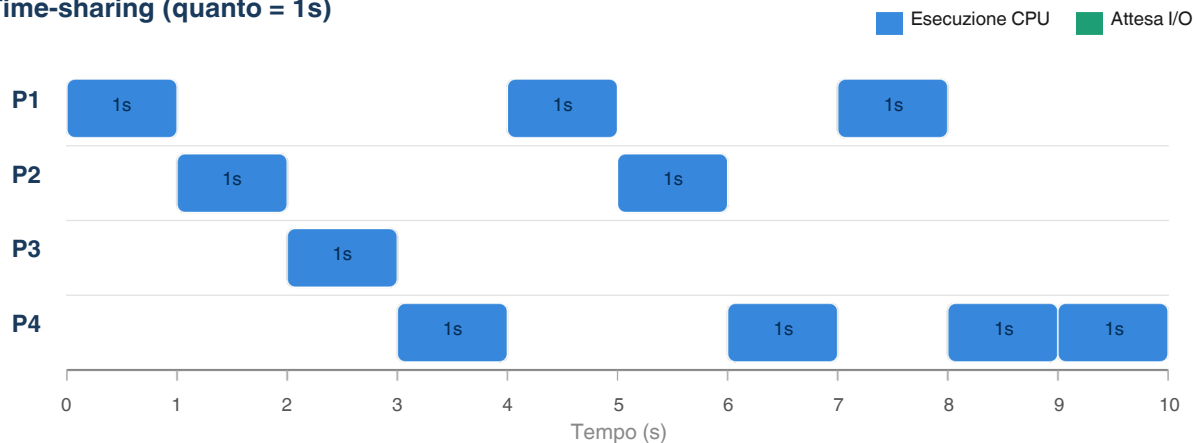


4.3 — Time-sharing con quanto = 1s (totale: ~10 secondi)

Ogni processo riceve 1 secondo di CPU in modo ciclico: P1, P2, P3, P4, P1, P2, P4, P1, P4, P4.

Nessun processo monopolizza la CPU ma ogni processo deve attendere il proprio turno.

Time-sharing (quanto = 1s)



5. Confronto dei Metodi

La tabella seguente confronta i tre metodi in base ai parametri più rilevanti per la gestione efficiente della CPU:

Parametro	Mono-tasking	Multi-tasking	Time-sharing
Tempo totale	15 secondi	11 secondi	~10 secondi
CPU idle	Si — durante I/O	No	No
Gestione I/O	CPU si ferma	CPU passa ad altro processo	Processo interrotto dopo il quanto
Complessita'	Bassa	Media	Alta (context switch)
Efficienza CPU	Bassa	Alta	Alta
Equita'	Bassa	Media	Alta (ogni processo ottiene il suo turno)

6. Metodo Scelto: Multi-tasking

Il metodo piu'efficace per la gestione dei processi P1, P2, P3, P4 e' il **Multi-tasking**. La scelta si basa sulle seguenti motivazioni:

Motivazione	Spiegazione
Eliminazione dell'idle	Nei dati forniti, P1 ha 2 secondi di I/O, P2 ha 1 secondo, P4 ha 1 secondo. Il mono-tasking sprecherebbe 4 secondi totali con la CPU idle. Il multi-tasking azzerà questo spreco.
Riduzione tempo totale	Il tempo totale scende da 15 secondi (mono-tasking) a 11 secondi: un miglioramento del 27% sulla stessa quantita' di lavoro.
Semplicità VS time-sharing	Il time-sharing introduce overhead di context switch e complessita' aggiuntiva. Con soli 4 processi e tempi definiti, il multi-tasking ottiene risultati simili con minore complessita' implementativa.
Adatto all'I/O bound	I processi di questo esercizio sono I/O bound (hanno attese esterne). Il multi-tasking e' progettato esattamente per questo scenario: sfruttare i periodi di inattivita' forzata per avanzare con altri processi.

7. Conclusioni

L'analisi dei tre metodi di scheduling ha dimostrato come la scelta dell'algoritmo di pianificazione influenzi significativamente le prestazioni di un sistema operativo. Il mono-tasking, pur essendo il piu' semplice, risulta inadeguato in presenza di processi I/O bound poiche' lascia la CPU inutilizzata per periodi non trascurabili. Il multi-tasking rappresenta la soluzione ottimale per questo set di processi: sfrutta i tempi di attesa I/O per eseguire altri processi, riducendo il tempo totale di completamento del 27% rispetto al mono-tasking, senza introdurre la complessita' aggiuntiva del time-sharing. Quest'ultimo, pur garantendo maggiore equita' tra i processi, risulta piu' adatto a scenari con molti processi concorrenti e utenti multipli.

Obiettivo	Stato
Analisi dei 3 metodi di scheduling	OK
Diagrammi di Gantt per ogni metodo	OK
Confronto tempi totali e efficienza CPU	OK
Identificazione metodo piu' efficace (Multi-tasking)	OK
Motivazione della scelta	OK